

Algorithmics	Student information	Date	Number of session
	UO: 307423	26/02	1.2
	Surname: Miranda Martín	 Escuela de Ingeniería Informática Universidad de Oviedo	
	Name: Mariana Patricia		



Activity 1. [Some iterative models]

All the following tables are measured in milliseconds and WITHOUT OPTIMIZATION (-Xint) in Java. We can type "OoT" for times over one minute.

After measuring times, fill in the table:

	$O(n)$	$O(n^2 \log n)$	$O(n^2 \log n)$	$O(n^3)$
N	tLoop1 (ms)	tLoop2 (ms)	tLoop3 (ms)	tLoop4 (ms)
100	0,05	0,03	0,12	1,02
200	0,09	0,1	0,46	7,64
400	0,2	0,49	1,99	59,4
800	0,44	2,14	8,39	475,69
1600	0,87	8,35	35,66	OoT
3200	1,91	36,24	152,9	OoT
6400	3,94	138,33	688,45	OoT
12800	1,97	619,03	OoT	OoT
25600	4,12	OoT	OoT	OoT
51200	8,68	OoT	OoT	OoT

Explain whether the different times obtained agree with what was expected, according to the theoretical complexity of the four cases.

As you can see based on the value of the table above, this activity compares how different complexity classes behave as the input size N increases:

Loop 1 shows linear growth, i.e., an $O(n)$ complexity. You can clearly see this when N doubles (e.g., from 400 to 800), the time roughly doubles (from 0.2 ms to 0.44 ms).

Both Loops 2 and 3 grow much faster than linear models but slower than cubic ones, i.e., they showcase a $O(n^2 \log n)$ complexity.

Loop 4 follows a cubic pattern, i.e., $O(n^3)$ complexity as the model reaches "OoT" (Over one minute) very quickly, becoming impractical for N values as small as 1,600.

Algorithmics	Student information	Date	Number of session
	UO: 307423	26/02	1.2
	Surname: Miranda Martín		
	Name: Mariana Patricia		

Activity 2. [Creation of iterative models of a given time complexity]

Implement three new classes Loop5.java, Loop6.java and Loop7.java, that simulate iterative algorithms with a complexity $O(n^2 \log 2n)$, $O(n^3 \log n)$ and $O(n^4)$ respectively. After implementing the classes, measure their execution times and fill in the table

N	tLoop5 (ms)	tLoop6 (ms)	tLoop7 (ms)
100	0,6	5,65	57,95
200	2,9	60,42	917,56
400	13,95	540,9	OoT
800	66,37	OoT	OoT
1600	312,86	OoT	OoT
3200	OoT	OoT	OoT
6400	OoT	OoT	OoT

Explain whether the different times obtained agree with what was expected, according to the theoretical complexity of the four cases.

All three classes behave as expected, i.e., their empirical performances correlate with their theoretical ones, being Loop7 clearly the slower one as it follows a $O(n^4)$ complexity.

Algorithmics	Student information	Date	Number of session
	UO: 307423	26/02	1.2
	Surname: Miranda Martín		
	Name: Mariana Patricia		

Comparison of two algorithms Environment

Activity 3. [A. Two algorithms with different complexity]

After measuring times, fill in the table:

N	tLoop1 (ms)	tLoop2 (ms)	t1/t2
100	0,05	0,03	1,667
200	0,09	0,1	0,900
400	0,2	0,49	0,408
800	0,44	2,14	0,206
1600	0,87	8,35	0,104
3200	1,91	36,24	0,053
6400	3,94	138,33	0,028
12800	1,97	619,03	0,003
25600	4,12	OoT	
51200	8,68	OoT	

Explain whether the different times and their quotient agree with what was expected according to the theoretical complexity.

As the values shown in the previous table clearly showcase, this activity highlights the increasing gap between a lower-complexity algorithm Loop 1, which follows a $O(n)$ complexity, and a higher-complexity one Loop 2, which grows following a $O(n^2 \log n)$ complexity.

As N increases, the quotient $t1/t2$ rapidly decreases almost reaching practically zero. This confirms that as the problem size grows, the efficiency of the algorithm's design (its complexity) becomes far more important than any initial speed advantages for small inputs.

Algorithmics	Student information	Date	Number of session
	UO: 307423	26/02	1.2
	Surname: Miranda Martín		
	Name: Mariana Patricia		

Activity 4. [B. Two algorithms with the same complexity]

After measuring times, fill in the table:

N	tLoop3(ms)	tLoop2 (ms)	t3/t2
100	0,12	0,03	4,00
200	0,46	0,1	4,60
400	1,99	0,49	4,06
800	8,39	2,14	3,92
1600	35,66	8,35	4,27
3200	152,9	36,24	4,22
6400	688,45	138,33	4,98
12800	OoT	619,03	
25600	OoT	OoT	
51200	OoT	OoT	

Explain whether the different times and their quotient agree with what was expected according to the theoretical complexity.

As it can be seen in the table above, this activity compares two different algorithms, Loop3 and Loop2, that share the same theoretical complexity, $O(n^2 \log n)$.

Although they have the same growth rate, Loop 3 is consistently 4 times slower than Loop 2. This shows that even when two algorithms grow at the same rate, internal differences in their implementation can lead to significant performance differences when putting them into practice.

Algorithmics	Student information	Date	Number of session
	UO: 307423	26/02	1.2
	Surname: Miranda Martín		
	Name: Mariana Patricia		

Activity 5. [C. Same algorithm in different development environments]

After measuring times, fill in the table:

N	tLoop4.py (ms)	tLoop4 without op (ms)	tLoop4 with op (ms)	t42/t41	t43/t42
100	0,4	1,02	0,5	2,55	0,49
200	3	7,64	1	2,55	0,13
400	23,7	59,4	5,8	2,51	0,10
800	203,7	475,69	43,3	2,34	0,09
1600	1770,9	OoT	334,8		
3200	18431	OoT	2631,9		
6400	OoT	OoT	OoT		

Explain whether the different times and their quotient agree with what was expected according to the theoretical complexity.

As the values and the head titles shown in the previous table clearly showcase, this activity compares the same program Loop 4, which follows a $O(n^3)$ complexity across different programming environments and settings.

If we compare the same program run both in Python and Java (No Optimization), the Python implementation is consistently faster than Java running without optimization. The quotient t_{42}/t_{41} is around 2.5, i.e., Loop4 executed in Java without optimization is 2.5 times slower than executed in Python.

Now, if we compare both Java executions, i.e., with and without optimization, the latter one is the fastest environment shown. By $N=800$, optimized Java is about 11 times faster than non-optimized Java.